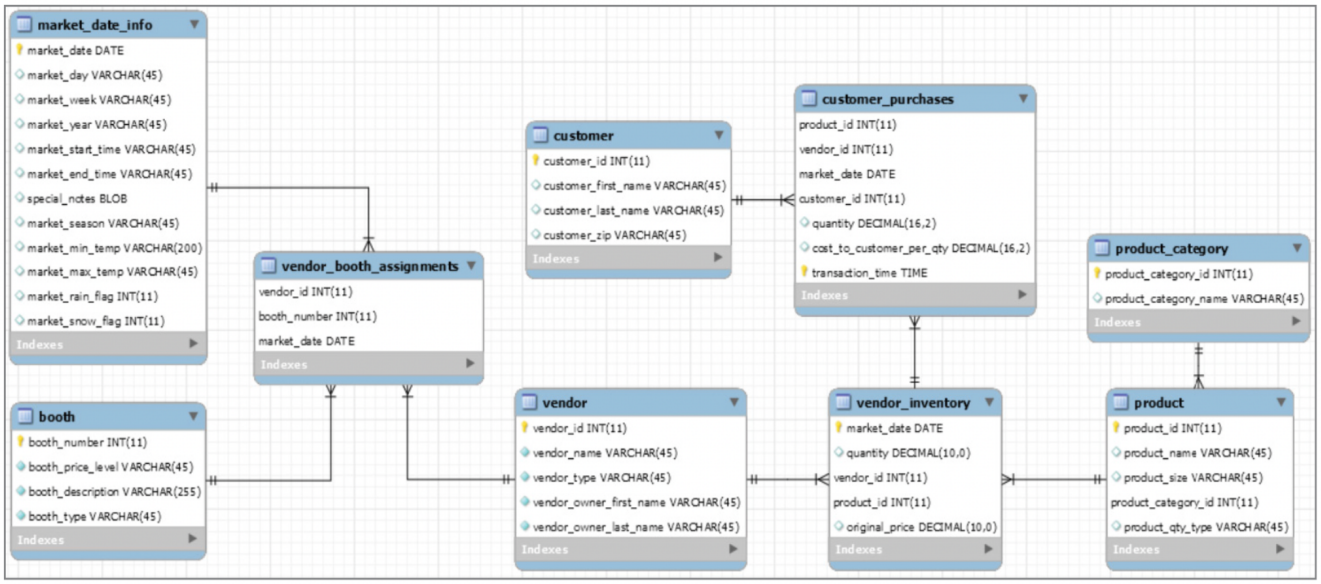
SQL - 03 - JOINS

Problem Statement:

You are a Data Analyst at the **Food Corporation of India** (FCI). You have been tasked to study the **Farmer's market - *Mandis***.

- Continuing with the same Farmer's market database.

Dataset: Farmer's Market Database



So far…

- You have learned to select data from **a single database table** and filter to the rows you want.
- BUT you might wonder what to do if the data you need exists across multiple related tables in the database.

Here's one analytical question for you

Q: Get details of all vendors selling fresh fruit or vegetables with product names and the quantity of that product in their inventory?

Now there are three types of information needed:

1. to filter to the categories with fresh fruit and vegetables - this will come from the ***product_category*** table.
2. to get details about each specific item, including product names and quantity types - the ***product*** table
3. to find out the quantity of the product for each vendor - the ***vendor_inventory*** table

So you need all this combined information from 3 tables here.

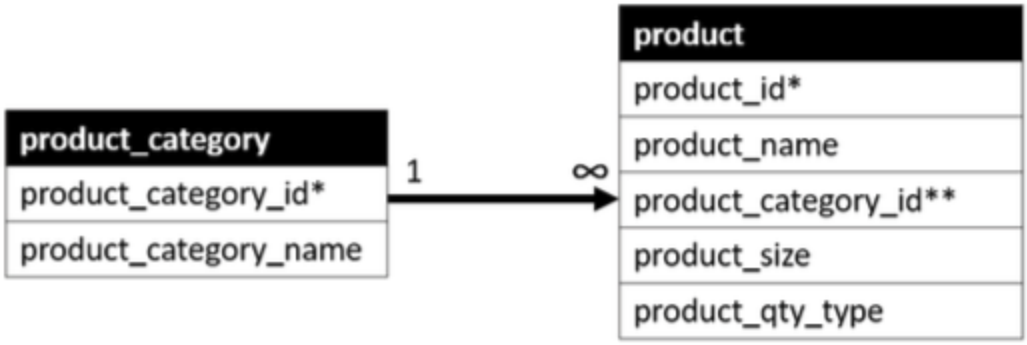
This is where **SQL JOINs** come in.

ER Diagrams are crucial in identifying which tables we can join and which key fields connect them.

Question: List all the products along with their product category name.

- Since only the ID of the product category exists in the ***product*** table, and the product category's name is in the ***product_category*** table,

- we **have to combine the data in the product and product_category tables together to generate this list.**



Now, there are multiple ways of joining these tables.

To combine these tables, we need to figure out which type of JOIN to use.

For the instructor: [You can pick your own example or watch the lecture to understand how to explain Joins]

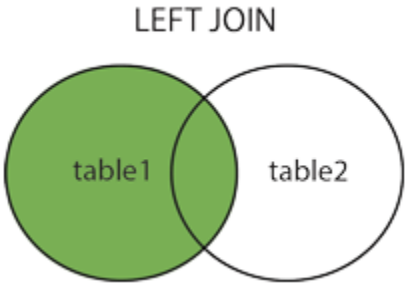
To illustrate the different types of SQL JOINs, we'll use the two tables from the Farmer's Market database but remove some columns to simplify the illustration.



- The figure shows the one-to-many relationship between these tables.
- Their primary keys are each identified with an asterisk and the foreign key with a double asterisk.
- Each row in the ***product_category*** table can be associated with many rows in the product table, but each row in the ***product*** table is associated with only one row in the ***product_category*** table.

- The fields that connect the two tables are *product_category.product_category_id* and *product.product_category_id*.

LEFT JOIN



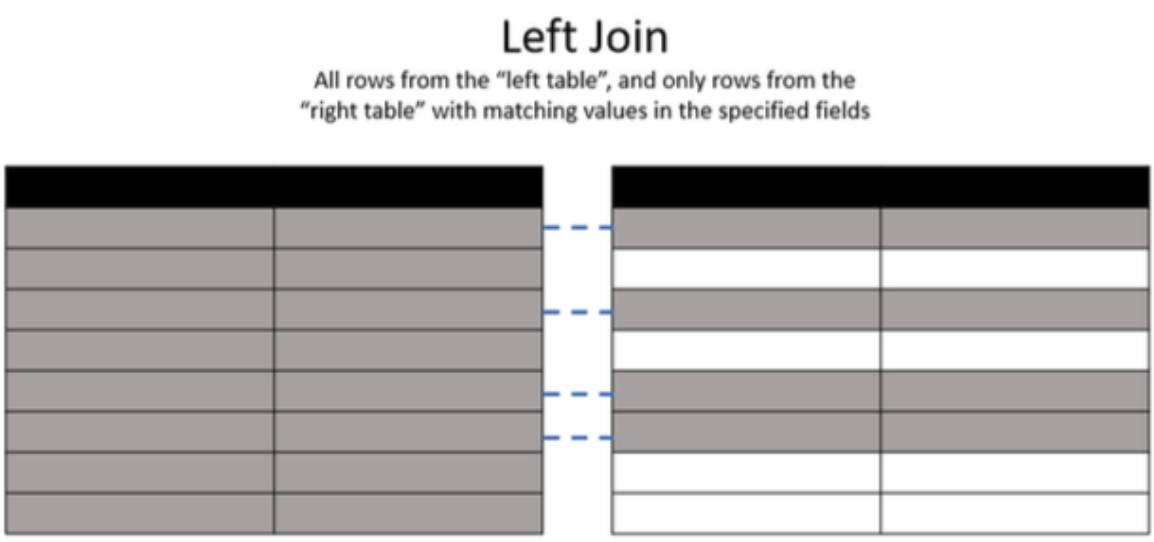
- This tells the DBMS to pull all records from the table on the “left side” of the JOIN, and only the matching records (based on the criteria specified in the JOIN clause) from the table on the “right side” of the JOIN.

[IMPORTANT]: How to decide which table is left and which table is right?

- As per our question we want to list **ALL** the products and their product categories. Which table should we use as left table if we are using LEFT JOIN?
- Ans: Product table should be on the left and product_categories should be on the right.

Syntax:

SELECT [columns to return]
FROM [left table]
[JOIN TYPE] [right table]
ON [left table].[field in left table to match] = [right table].[field in right table to match]



Actual query

```
SELECT * FROM
        product
LEFT JOIN product_category
        ON product.product_category_id = product_category.product_category_id
```

NOTE: You may have noticed two columns called **product_category_id** in the output.

That is because we selected all fields using the **asterisk(*)**, and **there are fields in both tables with the same name**.

To remedy this, **we could either specify the list of fields to be returned** and only include the **product_category_id** from one of the tables or **alias the column names to indicate which table each came from**.

Breaking down LEFT JOIN

- The LEFT JOIN indicates that we want all rows from the **product** table (which is listed on the left side of the JOIN keyword) and

- only the associated rows from the **product_category** table. So, if a category is not associated with any products, it will not be included in the results.

- If a product were without a category, it would be included in the results, with the fields on the **product_category** side being NULL.

- The ON part of the JOIN clause tells the query to match up the rows in the two tables using each table's values in the product_category_id field.

- We can specify which table each column is from since it's possible to have identically named columns in different tables.

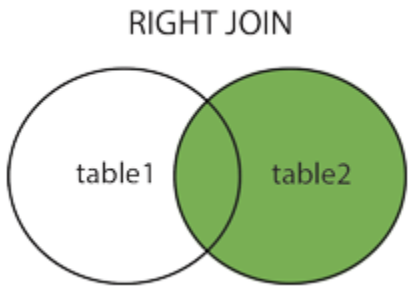
```
SELECT
        product.product_id,
        product.product_name,
        product.product_category_id AS product_prod_cat_id,
        product_category.product_category_id AS category_prod_cat_id,
        product_category.product_category_name
    FROM product
        LEFT JOIN product_category
            ON product.product_category_id = product_category.
    product_category_id
```

Another type of aliasing in a SQL query is for developer convenience because it isn't visible in the output: **table aliasing**.

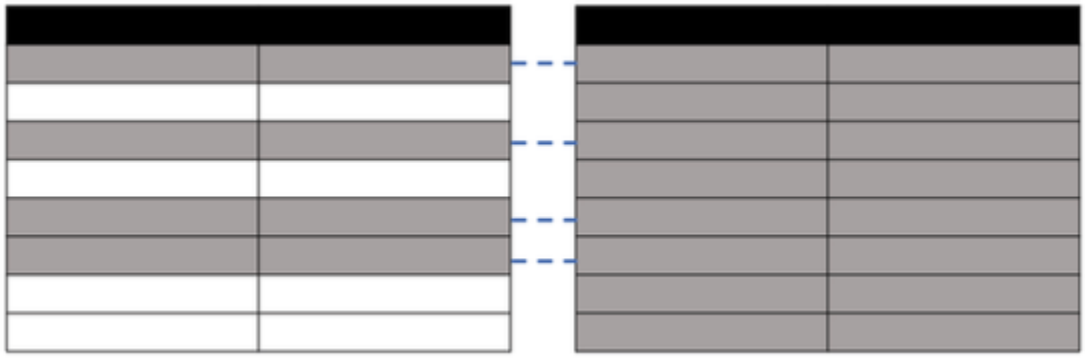
```
SELECT
        p.product_id,
        p.product_name,
        pc.product_category_id,
        pc.product_category_name
    FROM product AS p
```

```
        LEFT JOIN product_category AS pc
            ON p.product_category_id = pc.product_category_id
    ORDER BY pc.product_category_name, p.product_name
```

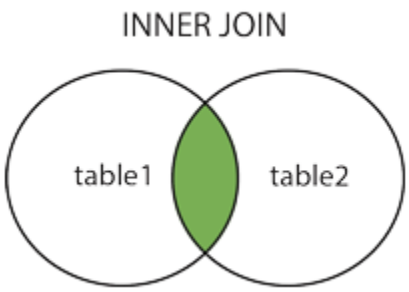
RIGHT JOIN



In a RIGHT JOIN, all of the rows from the “right table” are returned, along with only the matching rows from the “left table,” using the fields specified in the ON part of the query.



INNER JOIN



- An INNER JOIN only returns records that have matches in both tables.
- Can you tell which rows from each table will not be returned if we INNER JOIN the **product** and **product_category** tables on *product_category_id*?

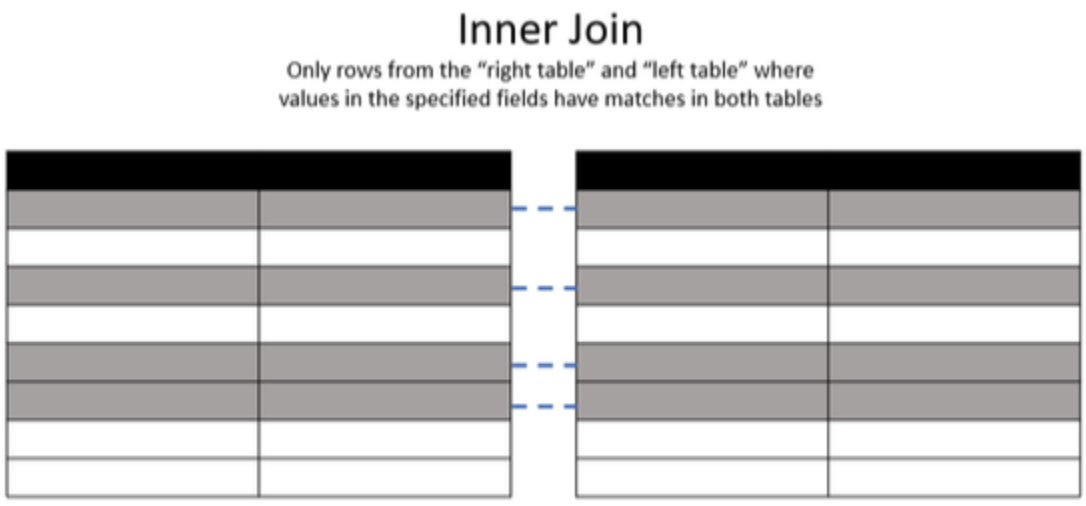


Figure 5.10

Question: Get all the Customers who have purchased nothing from the market yet.

What type of JOIN should we use here?

What if we do a LEFT JOIN here.

The two tables needed are:

1. Customer table
2. Customer_purchases table

```
SELECT *
  FROM customer AS c
  LEFT JOIN customer_purchases AS cp
      ON c.customer_id = cp.customer_id
```

- There can be customers without any purchases.
- Such customers were added to the customer table so we have their customer data, but they have not yet purchased any products.
- Since we did a LEFT JOIN, we're getting a list of all customers, and their associated purchases, if there are any.
- Customers with multiple purchases will show up in the output multiple times for each item purchased.
- Customers without purchases will have NULL values in all fields displayed from the customer_purchases table.

To get the list of customers that did not purchase anything

```
SELECT c.*  # select columns from customer table only
  FROM customer AS c
  LEFT JOIN customer_purchases AS cp
      ON c.customer_id = cp.customer_id
  WHERE cp.customer_id IS NULL
```

- Here we only selected columns from the customer table, using c.*,
- because all of the columns on the customer_purchases side of the relationship will be NULL (since we're filtering to NULL customer_id, and there are no purchases in the customer_purchases table without a customer_id, since it is a required field;)

This will give us all the customers who have not purchased anything yet.

Q: A simpler question: List all the customers and their associated purchases?

<How will you do this?>

Answer: RIGHT JOIN

```
SELECT *
  FROM customer AS c
  RIGHT JOIN customer_purchases AS cp
     ON c.customer_id = cp.customer_id
```

Output: there are no rows returned with NULL values in the customer table columns, because there is no such thing as a purchase without a customer_id.

And because we did a RIGHT JOIN, we will no longer get customers without purchases in the results.

Note: If you only want records from each table that have matches in both tables, use an INNER JOIN.

[Need to change the questions] Question: Write a query that returns a list of all customers who did not purchase on March 2, 2019.

We will use a LEFT JOIN since we want to include the customers who have never purchased at any farmer's market, so wouldn't have any records in the customer_purchases table:

[redacted]

Two problems with the output:

1. Some rows/ customers are missing because the market_date can also be NULL.
2. We are getting multiple rows for each customer which is not required.

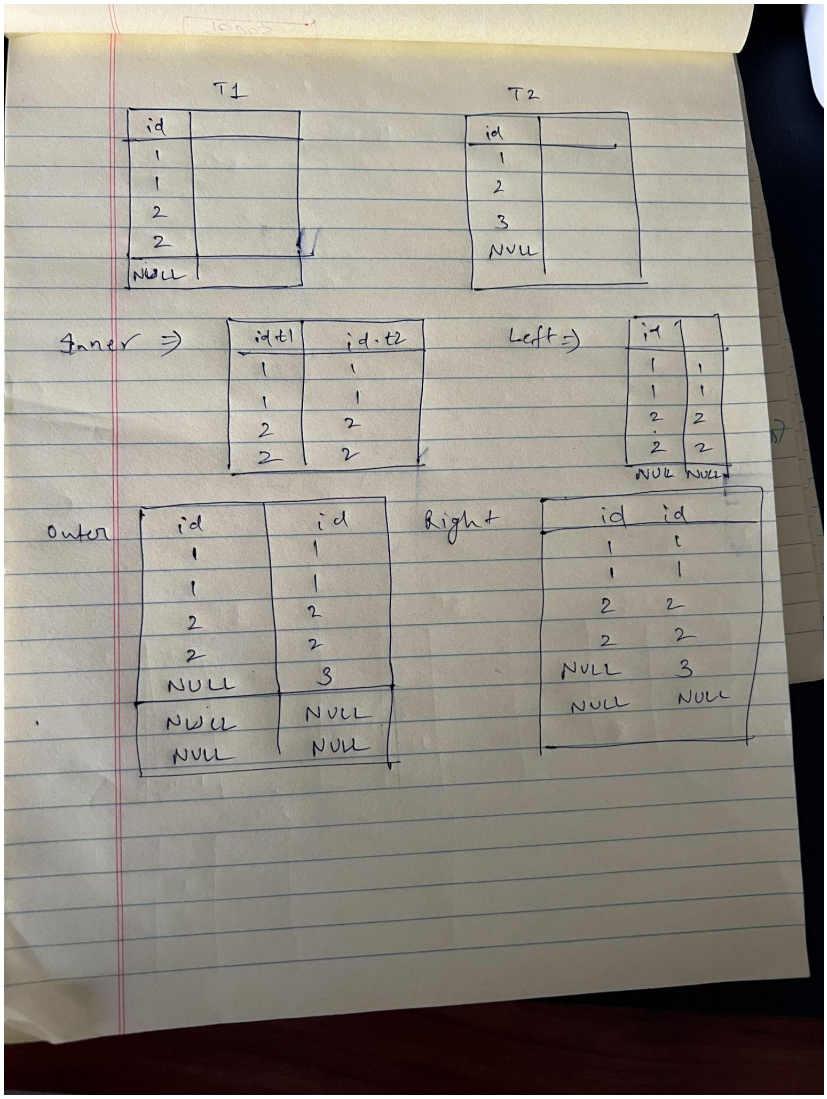
To solve first, we can add the following condition with an OR operator:

[redacted]

To solve 2nd, we can add a DISTINCT keyword that returns all the

[redacted]

Homework



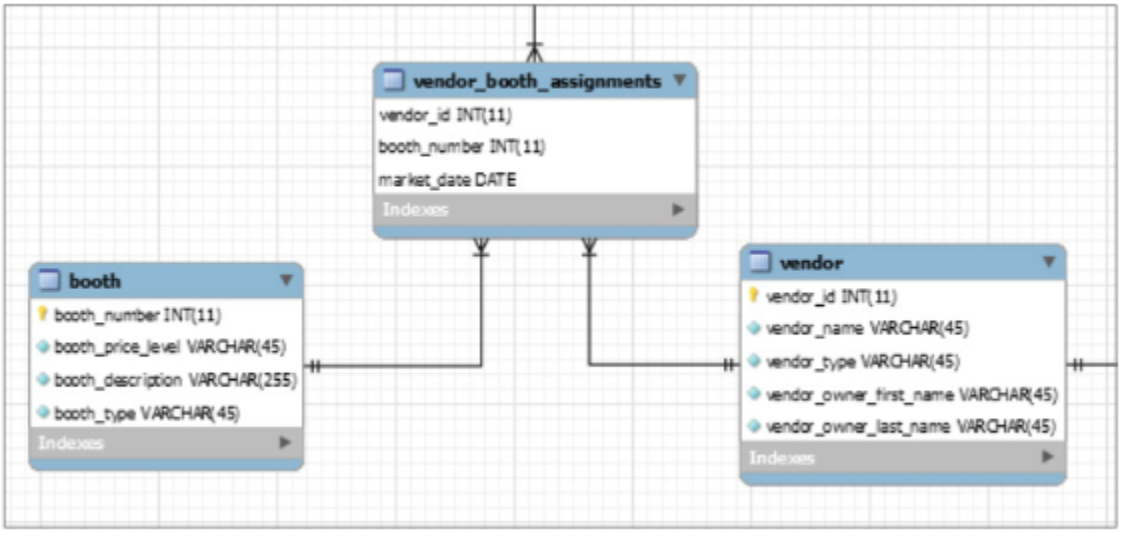
JOINs with more than two tables

Question: Let's say we want details about all farmer's market booths and every vendor booth assignment for every market date.

Imagine that you're building an interactive report that lets you filter to a booth, a vendor, or a date, to see the resulting list of booth assignments with additional booth and vendor details, so we need a merged dataset that contains all of their records.

This requires joining the three tables:

1. Vendor_booth_assignment
2. Booth
3. Vendor



Next question to solve this is: What kind of JOINs do you think we could use to ensure that all booths are included, even if they aren't assigned to a vendor yet, and all vendors assigned to booths are included?

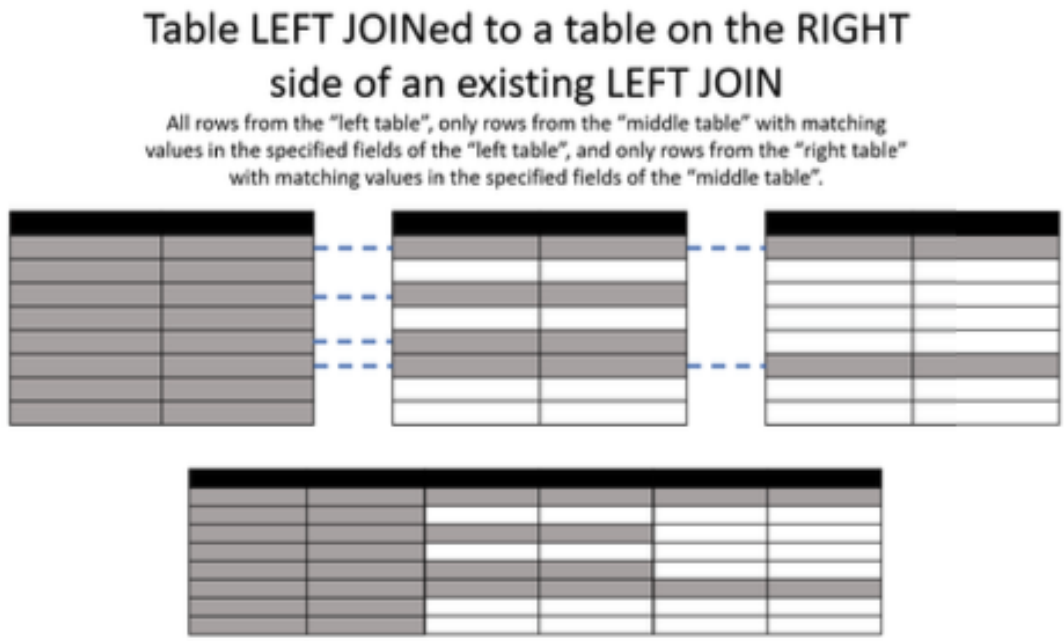
- **We can LEFT JOIN the vendor_booth_assignments to booth, therefore including all of the booths, and LEFT JOIN vendor to vendor_booth_assignments in the results**.

The query to accomplish these joins looks like this

<RUN ON Workbench>

```
SELECT
    b.booth_number,
    b.booth_type,
    vba.market_date,
    v.vendor_id,
    v.vendor_name,
    v.vendor_type
  FROM vendor_booth_assignments AS vba
    LEFT JOIN booth AS b
        ON vba.booth_number = b.booth_number
    LEFT JOIN vendor AS v
        ON v.vendor_id = vba.vendor_id
  ORDER BY b.booth_number, vba.market_date
```

- You can think of the second JOIN as being merged into the result of the first JOIN.
- Because in this case the *vendor_id* field in the third table, **vendor**, is joined to the *vendor_id* field in the second table, **vendor_booth_assignments**, only vendors that exist in the **vendor_booth_assignments** table will be included, resembling something like this:



What happens when you're trying to Join on a non-unique column?

You get duplicates in the output.

Table 5. Employees worked per day

Date	Employees Worked
2016-05-14	10
2016-05-15	15
2016-05-16	20
2016-05-17	25
2016-05-17	26

Date	Sales
2016-05-14	100
2016-05-15	200
2016-05-16	300
2016-05-17	400
2016-05-17	450

Table 6. Sales and employees worked per day

Date	Sales	Employees Worked
2016-05-14	100	10
2016-05-15	200	15
2016-05-16	300	20
2016-05-17	400	25
2016-05-17	400	26
2016-05-17	450	25
2016-05-17	450	26

SQL - 04 - Aggregating Results - Group BY

SQL starts becoming powerful for analysis when you use it to aggregate data.

Using the GROUP BY statement, you can specify the level of summarization and then use aggregate functions to summarize values for the records in each group.

- Data analysts can use SQL to build dynamic summary reports that can be automatically updated.

- Dashboards and reports built using software like Tableau and Cognos often rely on SQL queries to get the data they need from the underlying database in an aggregated form that can be used for reporting.

But it all starts with basic SQL aggregation.

GROUP BY Syntax

SELECT [columns to return]
FROM [table]
WHERE [conditional filter statements]
GROUP BY [columns to group on]
HAVING [conditional filter statements that are run after grouping]
ORDER BY [columns to sort on]

The GROUP BY keywords are followed by a comma-separated list of column names that indicate how you want to summarize the query results.

Important Points:

- GROUP BY clause is used with the SELECT statement.
- In the query, GROUP BY clause is placed after the WHERE clause.
- In the query, GROUP BY clause is placed before ORDER BY clause if used any.

Question: Get a list of the customer IDs of customers who made purchases on each market date.

If I write this…

```
SELECT
    market_date,
```

```
    customer_id
FROM farmers_market.customer_purchases
ORDER BY market_date, customer_id
```

- One row per item each customer purchased, **displaying duplicates in the output**, because you're querying the customer_purchases table with no grouping specified.

```
SELECT
    market_date,
    customer_id
FROM farmers_market.customer_purchases
GROUP BY market_date, customer_id
ORDER BY market_date, customer_id
```

- You can also accomplish the same result by using SELECT DISTINCT to remove duplicates,
- We are using GROUP BY to add summary columns to the output.

Aggregate functions - SUM() and COUNT()

Question: Count the number of purchases each customer made per market date.

```
SELECT
    market_date,
    customer_id,
    COUNT(*) AS num_purchases
FROM farmers_market.customer_purchases
GROUP BY market_date, customer_id
ORDER BY market_date, customer_id
LIMIT 10
```

Keep an eye on **granularity** of the table data

- The granularity of the **customer_purchases** table is such that if a customer were to buy three identical items, such as tomatoes, at once from a vendor, that would show up as 1 in the ***num_purchases*** column of this query's output, since the item purchase is recorded in one row in the table, with a quantity value of 3.
- If the customer were to buy three tomatoes, walk away from the stand, then go back and purchase another three tomatoes, that would be counted as two by the preceding query, since the new separate purchase would generate a new line in the database.

Alternate Question: Calculate the total quantity purchased by each customer per market_date.

Here, we'll use the SUM() function

```
SELECT
    market_date,
    customer_id,
    SUM(quantity) AS total_qty_purchased
 FROM farmers_market.customer_purchases
 GROUP BY market_date, customer_id
 ORDER BY market_date, customer_id
 LIMIT 10
```

- **Note** : It's important to **understand the granularity and structure** of the underlying table to ensure that your result means what you think it does.

- Thus, always write the query without aggregation first to see the values you will be summarizing before grouping the results.

Slightly complex question: how many different kinds of products were purchased by each customer on each market date:

Different kinds of products

```
SELECT
    market_date,
    customer_id,
    COUNT(DISTINCT product_id) AS different_products_purchased
 FROM farmers_market.customer_purchases
 GROUP BY market_date, customer_id
LIMIT 10
```

- So instead of counting how many rows there were in the customer_purchases table per customer per market date, like we did with COUNT(*),
- or adding up the quantities, like we did with SUM(quantity),
- we're identifying **how many unique product_id** values exist across those rows in the group

Add the qty summary to the same query:

```
SELECT
    market_date,
    customer_id,
    SUM(quantity) AS total_qty_purchased,
    COUNT(DISTINCT product_id) AS different_products_purchased
 FROM farmers_market.customer_purchases
 GROUP BY market_date, customer_id
 ORDER BY market_date, customer_id
```

Question: Calculate the total price paid by customer_id 3 per market_date.

Can perform Calculations Inside Aggregation Functions

```
SELECT
```

```
        customer_id,
        market_date,
        SUM(quantity * cost_to_customer_per_qty) AS total_spent
    FROM farmers_market.customer_purchases
    WHERE
        customer_id = 3
    GROUP BY market_date
    ORDER BY market_date
```

- The price will be calculated per row of the table, and then the results will be summed up per group.

- **Important :** Notice that **vendor_id has been removed from the list of columns** to be displayed and from the ORDER BY clause.
- That's because if we want the aggregation level of one row per customer per date, we **can't also include vendor_id** in the output, **because the customer can purchase from multiple vendors on a single date**, so the results wouldn't be aggregated at the level we wanted.

Question: What if we wanted to find out how much this customer had spent at each vendor, regardless of date?

Then we can group by customer_id and vendor_id:

```
SELECT
    customer_id,
    vendor_id,
    SUM(quantity * cost_to_customer_per_qty) AS total_spent
 FROM farmers_market.customer_purchases
 GROUP BY customer_id, vendor_id
 ORDER BY customer_id, vendor_id
```

Let's add some customer details and vendor details to these results. Customer details are in the customer table and vendor details are in the vendor table. - So that means we will have to JOIN

```
SELECT
    c.customer_first_name,
    c.customer_last_name,
    cp.customer_id,
    v.vendor_name,
    cp.vendor_id,
    cp.quantity * cp.cost_to_customer_per_qty AS price
FROM farmers_market.customer c
    LEFT JOIN farmers_market.customer_purchases cp
        ON c.customer_id = cp.customer_id
    LEFT JOIN farmers_market.vendor v
        ON cp.vendor_id = v.vendor_id
ORDER BY cp.customer_id, cp.vendor_id
```

- To summarize at the level of one row per customer per vendor, we will have to group by a lot more fields, including all of the customer table fields and all of the vendor table fields.
- Basically, we want to group by all of the displayed fields that don't include aggregate functions.

```
SELECT
    c.customer_first_name,
    c.customer_last_name,
    cp.customer_id,
    v.vendor_name,
    cp.vendor_id,
    ROUND(SUM(quantity * cost_to_customer_per_qty), 2) AS total_spent
FROM farmers_market.customer c
    LEFT JOIN farmers_market.customer_purchases cp
        ON c.customer_id = cp.customer_id
    LEFT JOIN farmers_market.vendor v
        ON cp.vendor_id = v.vendor_id
```

```
 WHERE
    cp.customer_id = 3
 GROUP BY
    c.customer_first_name,
    c.customer_last_name,
    cp.customer_id,
    v.vendor_name,
    cp.vendor_id
 ORDER BY cp.customer_id, cp.vendor_id
```

Question: We want to get the most and least expensive items per product category, considering the fact that each vendor sets their own prices and can adjust prices per customer.

We will use the **vendor_inventory** table, which has a field for the original price the vendors set for each item they bring to market on each market date.

First, let's look at all of the available fields in the vendor_inventory table by using select * query.

```
SELECT *
 FROM farmers_market.vendor_inventory
 ORDER BY original_price
 LIMIT 10
```

MIN and MAX

We can get the least and most expensive item prices in the entire table by using the MIN() and MAX() functions without grouping in MySQL

```
SELECT
    MIN(original_price) AS minimum_price,
    MAX(original_price) AS maximum_price
 FROM farmers_market.vendor_inventory
 ORDER BY original_price
```

- But if we want to **get the lowest and highest prices within each product category**, we have to group by the product_category_id,
- then the summary values will be calculated per group,

```
SELECT
    pc.product_category_name,
    p.product_category_id,
    MIN(vi.original_price) AS minimum_price,
    MAX(vi.original_price) AS maximum_price
 FROM farmers_market.vendor_inventory AS vi
    INNER JOIN farmers_market.product AS p
        ON vi.product_id = p.product_id
    INNER JOIN farmers_market.product_category AS pc
        ON p.product_category_id = pc.product_category_id
 GROUP BY pc.product_category_name, p.product_category_id
```

- Get prices from vendor_inventory
- Products from the product table
- Their Categories from the product_category table.

Question: Count how many products were for sale on each market date, or how many different products each vendor offered.

We can determine these values using COUNT and COUNT DISTINCT.

COUNT will count up the rows within a group when used with GROUP BY, and COUNT DISTINCT will count up the unique values present in the specified field within the group.

- To determine how many products are offered for sale each market date, we can count up the rows in the **vendor_inventory** table, grouped by date.
- This doesn't tell us what quantity of each product was offered or sold, but counts the number of products available, because there is a row in this table for each product for each vendor for each market date.

```
SELECT
    market_date,
    COUNT(product_id) AS product_count
 FROM farmers_market.vendor_inventory
 GROUP BY market_date
 ORDER BY market_date
```

If we wanted to know how many different products—with unique product IDs—each vendor brought to market during a date range, we could use COUNT DISTINCT on the product_id field, like so:

```
SELECT
    vendor_id,
    COUNT(DISTINCT product_id) AS different_products_offered
 FROM farmers_market.vendor_inventory
 WHERE market_date BETWEEN '2019-03-02' AND '2019-03-16'
 GROUP BY vendor_id
ORDER BY vendor_id
```

Question: In addition to the count of different products per vendor, we also want the **average original price of a product per vendor**?

We can add a line to the preceding query, and use the AVG() function

```
SELECT
    vendor_id,
    COUNT( product_id) AS different_products_offered,
    AVG(original_price) AS average_product_price
 FROM farmers_market.vendor_inventory
 WHERE market_date BETWEEN '2019-03-02' AND '2019-03-16'
 GROUP BY vendor_id, product_id
 ORDER BY vendor_id
```

But is this average product price??

- Is it fair to call it “average product price” when the underlying table has one row per type of product?
- If the vendor brought 100 tomatoes to market, those would all be in one line of the underlying vendor inventory table, so the price of a tomato would only be included in the average once.
- If you calculated the “average product price” for the vendor this way, you would just get the average of the price of one tomato and one bouquet.

How to calculate the price per item?

- To get an **actual average price of items in each vendor’s inventory** between the specified dates, it might make more sense to **multiply the quantity** of each type of item times the price of that item, which is a calculation that would occur per row, then sum that up and divide by the total quantity of items, which is a calculation that would occur per vendor.

- Let’s try a calculation that includes these two summary values.

```
SELECT
    vendor_id,
    COUNT(DISTINCT product_id) AS different_products_offered,
    SUM(quantity * original_price) AS value_of_inventory,
    SUM(quantity) AS inventory_item_count,
    ROUND(SUM(quantity * original_price) / SUM(quantity), 2) AS
```

```
 average_item_price
 FROM farmers_market.vendor_inventory
 WHERE market_date BETWEEN '2019-03-02' AND '2019-03-16'
 GROUP BY vendor_id
 ORDER BY vendor_id
```

Filtering with Having

Filtering is another thing that can be done in the query after summarization occurs.

If you want to filter values after the aggregate functions are applied, you can add a HAVING clause to the query. This filters the groups based on the summary values.

Question: filter out vendors who brought at least 10 items to the farmer's market over the time period - 2019-05-02 and 2019-05-16

```
SELECT
    vendor_id,
    COUNT(DISTINCT product_id) AS different_products_offered,
    SUM(quantity * original_price) AS value_of_inventory,
    SUM(quantity) AS inventory_item_count,
    SUM(quantity * original_price) / SUM(quantity) AS average_item_price
 FROM farmers_market.vendor_inventory
 WHERE market_date BETWEEN '2019-03-02' AND '2019-03-16'
 GROUP BY vendor_id
 HAVING inventory_item_count >= 100
 ORDER BY vendor_id
```

TIP: If you GROUP BY all of the fields that are supposed to be distinct in your resulting dataset, then add a HAVING clause that filters to aggregated rows with a COUNT(*) > 1, any results returned indicate that there is more than one row

with your “unique” combination of values, highlighting the existence of unwanted duplicates in your database or query results!